

E-Comm Review Analyzer

Una solución integral para el análisis de sentimiento y visualización estadística de datos de comercio electrónico



Lijia Gong

01/2026

INTRODUCCIÓN

1. Introducción y Objetivos

1.1. Contexto y Motivación del Proyecto

Hoy en día, el comercio electrónico ha cambiado por completo nuestra forma de comprar. Al no poder tocar los productos físicamente, las opiniones de otros compradores se han vuelto esenciales para decidir si añadimos algo al carrito.

Estas reseñas son una "mina de oro" de información. En ellas, los clientes explican si la talla es pequeña, si la tela es de buena calidad o si el diseño cumple las expectativas. Sin embargo, cuando una tienda recibe miles de comentarios, es imposible leerlos todos a mano. Aquí es donde surge este proyecto: la necesidad de crear una herramienta que procese automáticamente estos textos y los convierta en información útil para el negocio.

Para este trabajo, hemos utilizado el dataset "*Women's E-Commerce Clothing Reviews*", que combina calificaciones numéricas, categorías de ropa y, lo más importante, las opiniones escritas de las usuarias.

Nota sobre el uso de IA: El desarrollo de este código ha contado con el apoyo de herramientas de Inteligencia Artificial para tareas de depuración y mejora, siguiendo los criterios éticos de la CEI. El registro de estas interacciones se encuentra en el Anexo: Registro de Prompts de IA.

1.2. Objetivos del Proyecto

El objetivo principal es desarrollar una herramienta en Python que convierta miles de opiniones de clientes en información visual y estratégica para el negocio.

Para lograrlo, se han definido los siguientes puntos:

- **Estructura Modular:** Crear un paquete organizado en tres módulos independientes para limpiar datos, generar gráficos y analizar textos.
- **Visualización e Interactividad:** Diseñar un panel de control en Streamlit con gráficos y mapas de calor que permitan explorar los datos sin saber programar.

-
- **Análisis de Sentimiento y Temas:** Identificar automáticamente si las críticas son positivas o negativas y extraer las palabras más repetidas mediante técnicas de NLP.

2. Diseño y Desarrollo

2.1. Arquitectura del paquete

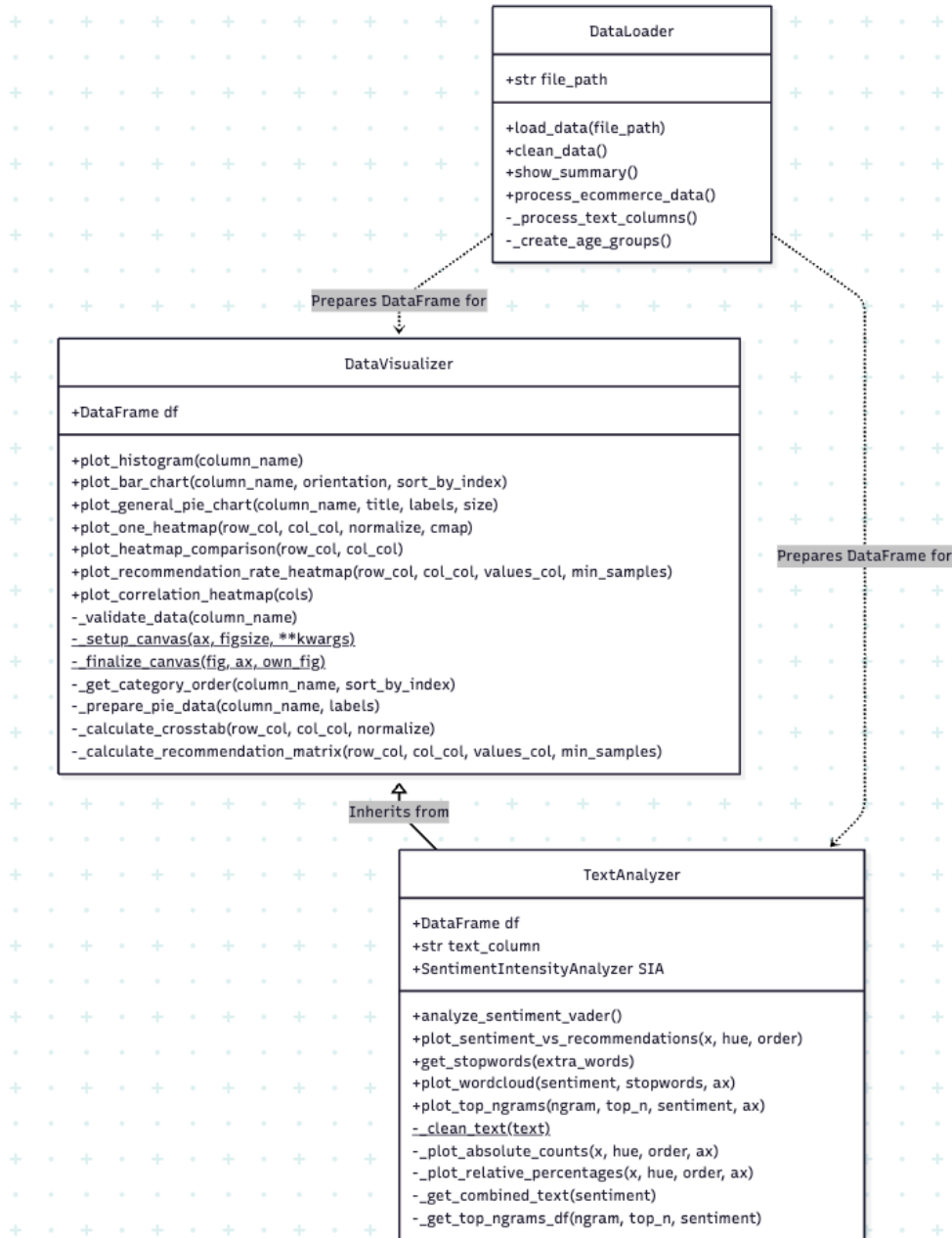
El proyecto se ha diseñado siguiendo el paradigma de Programación Orientada a Objetos (POO). Se ha optado por una arquitectura modular basada en clases. Cada clase tiene una responsabilidad única, lo que facilita la depuración y la futura expansión del código.

La estructura del paquete *ecomm_review_analyzer* se divide en 3 componentes principales:

1. Capa de Gestión de Datos (Clase *DataLoader*): Responsable de la limpieza de datos crudos.
2. Capa de Visualización (Clase *DataVisualizer*): Encargada de transformar los datos procesados en representaciones gráficas estáticas y estadísticas.
3. Capa de Análisis de reseñas (Clase *TextAnalyzer*): Especializada en el procesamiento de lenguaje natural (NLP) y la visualización de WordCloud.

Diagrama UML:

La generación del diagrama UML se realizó mediante la plataforma **Mermaid** (<https://mermaid.js.org/>).



2.3. Estructura de Directorios

La organización física del proyecto refleja esta separación lógica:

Project Structure

```
ecomm_review_analyzer/
├── ecomm_review_analyzer/      # Main Source Code Package
│   ├── __init__.py            # Package initialization
│   ├── data_loader.py         # Logic for loading, cleaning, and transforming data
│   ├── data_visualizer.py     # Logic for statistical plotting (Matplotlib/Seaborn)
│   ├── text_analyzer.py       # Logic for NLP, VADER sentiment analysis, and N-grams
│   ├── main.py                # main
│   └── errors.py              # Custom exception definitions for error handling
├── data/                      # Data Directory
│   └── Womens Clothing...csv   # Raw dataset (usually excluded from version control)
├── docs/                      # Extended Documentation
│   ├── en/INSTRUCTIONS.md     # Detailed setup instructions in English
│   └── es/INSTRUCTIONS.md     # Detailed setup instructions in Spanish
├── app.py                     # Streamlit Application (Web Dashboard entry point)
├── plot.ipynb                 # Jupyter Notebook (Used for prototyping/EDA)
├── requirements.txt            # Production dependencies (for end-users)
├── requirements_dev.txt        # Development dependencies (for contributors/testing)
├── README.MD                  # Documentation
├── setup.py                   # Installation script for the package
├── setup.cfg                  # Static configuration for package metadata
├── pyproject.toml             # Modern build system configuration
├── Makefile.txt               # Automation shortcuts (install, test, clean)
├── CHANGELOG.md               # History of changes and versions
├── VERSION.txt                # Single source of truth for the project version
└── LICENSE.txt                # MIT License terms
```

3. Código Fuente

El proceso de carga y preprocesamiento de datos constituye un paso fundamental en el desarrollo del proyecto, ya que garantiza la calidad y consistencia de los datos utilizados en los análisis posteriores. Este capítulo describe las etapas involucradas en la obtención, carga, preparación de los datos provenientes del dataset elegido y la visualización.

3.1. Obtención y formato de los datos

Los datos utilizados en este proyecto fueron obtenidos de una fuente confiable llamada *Kaggle*, que ofrece este dataset *Women's E-Commerce Clothing Reviews*. Este dataset del sector de comercio electrónico de ropa femenina se centra en las reseñas escritas por los clientes. Sus nueve características de apoyo ofrecen un entorno ideal para analizar el texto a través de sus múltiples dimensiones. Según el autor de tal dataset, dado que se trata de datos comerciales reales, se han anonimizado y las referencias a la empresa en el texto y el cuerpo de la reseña se han reemplazado por "minorista" (Agarap, 2018).

Contenido:

Este *dataset* incluye 23486 filas y 10 variables de características. Cada fila corresponde a una reseña de cliente e incluye las siguientes variables (Agarap, 2018).:

- **Clothing ID**: Variable categórica entera que se refiere a la prenda específica que se reseña.
- **Age**: Variable entera positiva que indica la edad del revisor.
- **Title**: Variable de cadena para el título de la reseña.
- **Review Text**: Variable de cadena para el cuerpo de la reseña.
- **Rating**: Variable entera ordinal positiva que indica la puntuación del producto otorgada por el cliente, de 1 (Pésimo) a 5 (Excelente).
- **Recommended IND**: Variable binaria que indica si el cliente recomienda el producto, donde 1 es recomendado y 0 es no recomendado.
- **Positive Feedback Count**: Entero positivo que documenta el número de otros clientes que encontraron esta reseña positiva.
- **Division Name**: Nombre categórico de la división principal del producto.
- **Department Name**: Nombre categórico del departamento del producto.
- **Class Name**: Nombre categórico de la clase del producto.

3.2. Preprocesamiento de los datos

La carga de los datos se realizó utilizando la biblioteca Pandas, ampliamente conocida por su eficiencia en la manipulación de grandes cantidades de datos.

Limpieza genérica y resumen de los datos originales

Para generalizar el uso de este paquete, se desarrollaron los métodos *clean_data* y *show_summary* en la clase *DataLoader*. Su objetivo es eliminar la columna “*Unnamed: 0*” (si existe en el conjunto de datos) y mostrar un breve resumen sobre los tipos de datos, la cantidad de valores únicos, y valores nulos de cada columna, facilitando así los siguientes pasos de la limpieza (Pandas, 2026).

Identificación y manejo de valores nulos

Se destacaron valores nulos en columnas clave como *Title* y *Review Text*. Para abordar este problema, se desarrolló un método en la clase *DataLoader* denominado *process_ecommerce_data*, lo cual realiza las siguientes tareas:

-
- Elimina las filas con valores nulos en la columna *Review Text*. Dado que el objetivo principal es analizar reseñas, los registros que carecen de texto no aportan valor al análisis, por lo que la opción óptima es descartarlos.
 - Rellena los valores faltantes de la columna *Title* con una cadena vacía ("").
 - Genera una nueva columna combinando el contenido de *Title* y *Review Text*, denominada *title_with_review*.
 - Transforma una variable continua (*Age*) en una categórica (*Age Group*), un paso esencial para los análisis demográficos posteriores.

3.4. Análisis Exploratorio y Visualización (Clase *DataVisualizer*)

Después de la limpieza del conjunto de datos, comienza el proceso de Análisis Exploratorio de Datos (EDA) utilizando la clase *DataVisualizer*. Esta clase se basa principalmente en librerías como *matplotlib*, *seaborn* para la visualización, generando gráficos como histogramas (*histograms*), gráficos circulares (*pie charts*), gráficos de barras (*bar charts*) y mapas de calor (*heatmaps*), los cuales son esenciales para comprender la distribución de los datos.

Mediante el método protegido `_validate_data`, se valida la existencia de la columna en el dataset antes de generar los gráficos. Esto aumenta la robustez del código y es esencial para evitar errores de ejecución derivados de nombres de columna incorrectos.

Y luego se destacan las principales funciones de visualización y la lógica detrás de ellas:

Distribución de Variables Numéricas

Se desarrolló el método `plot_histogram` mayoritariamente para examinar la distribución de variables como la edad de los clientes o el conteo de *feedback* positivo (*Positive Feedback Count*) (ver ref. 1.1 en Anexo de Prompts). Sus características principales son (Austin Chia, 2024):

- **KDE:** Se utilizó `sns.histplot` con Estimación de Densidad de Kernel (KDE) para visualizar la forma de la distribución de forma más suavizada y continua.
- **Legibilidad:** En caso de que el eje X contenga cadenas de texto, las etiquetas se rotan 45 grados para evitar superposiciones y facilitar la lectura.
- **Uso de Subplots:** Se introdujo el parámetro `ax` (por defecto `None`). Esto permite renderizar múltiples gráficos en una misma fila (pasando, por ejemplo, `ax=ax[0]` o `ax=ax[1]`), lo cual ahorra espacio y facilita la comparación visual.

-
- **Reusabilidad:** El nombre de la columna (*column_name*) se define como un parámetro, permitiendo aplicar la función a diferentes variables numéricas del dataset.

Análisis de Frecuencia Categórica

Para entender qué categorías de productos dominan el inventario, se desarrolló el método *plot_bar_chart* (ver ref. 1.2 en Anexo de Prompts) (GeeksforGeeks, 2025).

- **Ordenamiento flexible:** El método incluye un parámetro flexible *sort_by_index*. Esto permite visualizar los datos ordenados por volumen(frecuencia descendente) o por orden lógico (alfabético), lo cual fue crucial para visualizar correctamente los grupos de edad (*Age Group*).
- **Orientación flexible:** Se introdujo el parámetro *orientation* para adaptar el gráfico según la longitud de las etiquetas.
 - Para cadenas de texto largas (como *Division Name*, *Department Name* y *Class Name*), es conveniente usar la orientación horizontal (h).
 - Para etiquetas cortas o numéricas (como *Age Group*, *Rating* y *Recommended IND*), resulta más adecuado utilizar la orientación vertical (v).
- Al igual que en el histograma, se mantuvieron los parámetros *column_name* y *ax* para garantizar la modularidad y la capacidad de crear figuras compuestas.

Mapas de Calor (Heatmap)

Una de las funcionalidades más avanzadas del paquete es la capacidad de cruzar variables mediante mapas de calor, a través de los métodos *plot_one_heatmap*, *plot_heatmap_comparison* y *plot_recommendation_rate_heatmap* (ver ref. 1.3 en Anexo de Prompts).

- *plot_one_heatmap*: Utiliza el parámetro *normalize*. Si *normalize=False*, se genera un *DataFrame* de frecuencias absolutas mediante *pd.crosstab*. En caso contrario (*normalize=True*), se calculan los porcentajes, facilitando la comparación relativa entre categorías (Seaborn, 2024).
- *plot_heatmap_comparison*: Al introducir dos columnas para comparar, esta función genera automáticamente dos mapas de calor en la misma fila basándose en la función anterior *plot_one_heatmap*: el de la izquierda muestra la frecuencia absoluta y el de la derecha la distribución porcentual.
- *plot_recommendation_rate_heatmap*: A diferencia de las anteriores, esta función introduce una tercera columna como variable (*values_col*). Utiliza *df.pivot_table* para determinar, por ejemplo, qué grupo de edad en qué categoría de ropa tiene un mayor porcentaje promedio de recomendación (Bothma, 2024) (NICAPOTATO, 2019).

-
- **Filtro Estadístico:** En el análisis de tasas de recomendación, se aplicó una lógica de filtrado (*rate_table.where(count_table >= min_samples)*) para ocultar aquellos segmentos con muestras insuficientes (por ejemplo, menos de 5 registros), evitando así conclusiones engañosas basadas en datos escasos.

3.5. Análisis de Sentimiento y Procesamiento de Lenguaje Natural (Clase *TextAnalyzer*)

Con el fin de realizar un análisis de Procesamiento de Lenguaje Natural y expandir la utilidad del paquete, se desarrolló la clase *TextAnalyzer*. Esta clase hereda directamente de *DataVisualizer*, una decisión de diseño que permite aprovechar las visualizaciones gráficas preexistentes y evitar la duplicación de código.

El análisis de texto se estructuró en tres fases clave:

Limpieza de texto

Antes de aplicar cualquier algoritmo de NLP, se implementó el método estático (*@staticmethod*) *_clean_text* para limpiar la columna de texto (ver ref. 2 en Anexo de Prompts).

- **Metodología:** Se utilizó la librería *BeautifulSoup* para eliminar etiquetas HTML y expresiones regulares para quitar URLs y espacios redundantes (Atif Ali AK, 2025).
- **Motivo:** Este paso es crítico para evitar que el ruido (como enlaces, que es lo habitual que se aparece en las reseñas) moleste la frase del cálculo de frecuencias de palabras y la clasificación de sentimientos.

Clasificación de Sentimiento con VADER

En vez de entrenar un modelo de Machine Learning para hacer el análisis de sentimiento, se optó por integrar la librería existente de Python: VADER (Valence Aware Dictionary and Sentiment Reasoner) (ver ref. 3 en Anexo de Prompts).

- **Metodología:** Se calculó el *compound score* (puntuación compuesta) para cada reseña mediante las funciones integradas de VADER.
- **Clasificación:** Basándose en esta puntuación compuesta, se clasificaron las reseñas en tres segmentos:
 - Positiva ($\text{score} \geq 0.05$),
 - Negativa ($\text{score} \leq -0.05$)
 - Neutral ($-0.05 < \text{score} < 0.05$)

Visualización Semántica

Para profundizar en el análisis, se desarrollaron métodos específicos orientados a identificar los términos más recurrentes en las reseñas positivas y negativas. Esto permite discernir si factores como la textura, la talla o el diseño son los determinantes de la satisfacción del cliente (ver ref. 1.4 y 1.5 en Anexo de Prompts).

- *plot_sentiment_vs_recommendations*: Se diseñó un análisis dual que combina un gráfico de conteo absoluto con uno de barras porcentuales (García-Ferrerira, 2025). El primero muestra el volumen total de reseñas por sentimiento, mientras que el segundo visualiza la proporción de recomendaciones dentro de cada categoría.

Esto permitió validar una hipótesis clave: ¿implica siempre un sentimiento positivo una recomendación de compra?

- *plot_wordcloud*: Para la generación de nubes de palabras segmentadas, se destaca la implementación del método auxiliar *get_stopwords* (Lakshmipathi N, 2021). Esta permite la inclusión de un parámetro *extra_words* para filtrar términos que, aunque frecuentes, carecen de valor semántico para el análisis de sentimiento en este contexto específico (ej. "dress", "petite", "made", "will") (Mike Chirico, 2016) (Graph Gallery, 2024).
- *plot_top_ngrams*: Se implementó un análisis de frecuencia de N-gramas (unigramas, bigramas o trigramas) utilizando *CountVectorizer* de la librería *scikit-learn*. Esta técnica permitió identificar no solo palabras aisladas, sino conceptos compuestos (ej. "runs small", "poor quality") que a menudo resultan invisibles en una nube de palabras tradicional pero que son cruciales para entender el contexto (Ranjith kumar G, 2020).

3.6 Interfaz y Flujo de Trabajo (app.py)

Para que los análisis técnicos sean accesibles y fáciles de usar, desarrollé una aplicación web con **Streamlit**. El archivo *app.py* conecta las tres clases (*DataLoader*, *DataVisualizer*, *TextAnalyzer*) en un proceso lógico paso a paso.

La estructura base de este *app.py* y sus métodos fueron generados mediante asistencia de IA para agilizar el desarrollo en la parte de la estructura. El código resultante fue revisado y adaptado manualmente (ver ref. 1.6 en Anexo de Prompts).

El diseño se centra en la facilidad de uso:

Carga de Datos

La aplicación aprovecha el ancho completo de la pantalla. Desde la barra lateral, el usuario sube el archivo CSV y el *DataLoader* se activa al instante: limpia los datos y muestra una vista previa de la tabla original y la procesada.

Gráficos a la Carta

Una vez limpios los datos, el usuario tiene el control:

- **Selección Flexible:** Usé *st.multiselect* para que el usuario elija exactamente qué columnas quiere analizar (ej. *Age*, *Feedback*).
- **Diseño Automático:** El código ajusta el diseño automáticamente. Usando un bucle y *math.ceil*, los gráficos se organizan ordenadamente en filas de dos, sin importar cuántas variables se seleccionen.
- **Ajustes Inteligentes:** Si la columna tiene datos muy sesgados (como el "*Feedback*"), la app aplica sola una escala logarítmica para que el gráfico se entienda mejor.

Panel de Texto y Métricas

La parte final usa el *TextAnalyzer* para dar sentido a las reseñas escritas.

- **Resumen Rápido:** Puse métricas grandes (KPIs) arriba para ver de un vistazo la nota media y el sentimiento general.
- **Explorador de Reseñas:** Añadí un filtro (*st.selectbox*) para ver las opiniones de un departamento específico. Esto permite leer las quejas reales de los clientes junto a la puntuación que les dio el algoritmo.

La creación de la interfaz web sirvió para confirmar que todo el código funciona bien en la práctica:

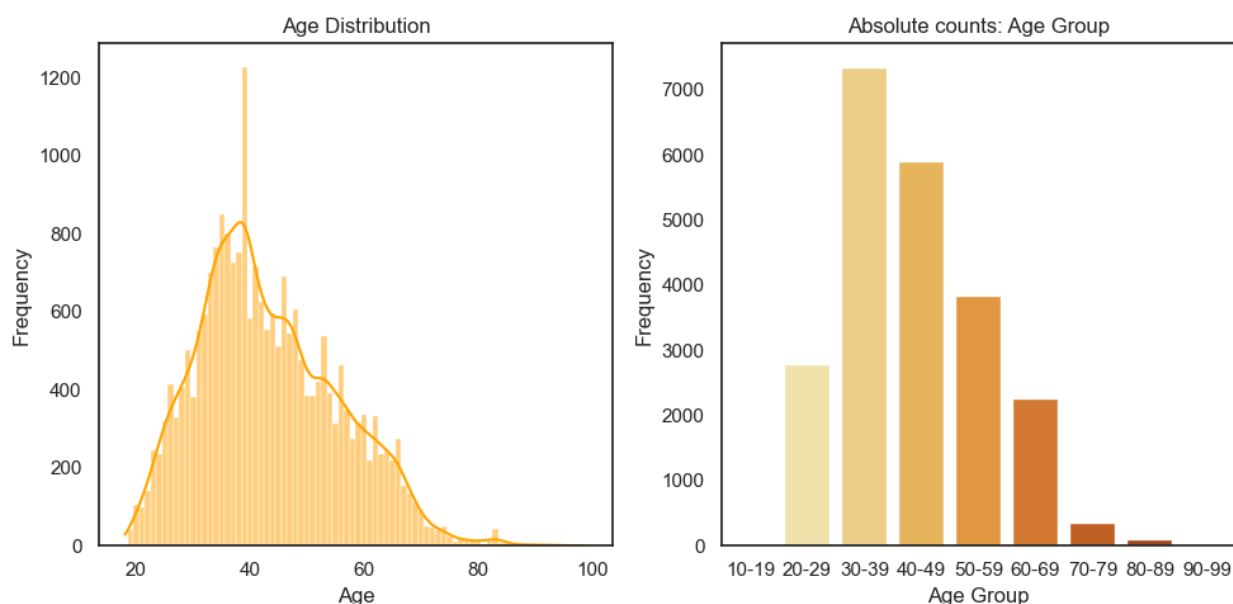
- **Estabilidad:** La app carga archivos y procesa datos pesados sin colgarse, avisando al usuario cuando está "pensando" (con *st.spinner*).
- **Gráficos Claros:** Los gráficos de seaborn y matplotlib se ven perfectos en la web, respetando los estilos que define en las clases.
- **Utilidad Real:** La función de filtrar reseñas por departamento demostró ser muy útil. Permite pasar de ver un "número rojo" en un gráfico a leer la razón exacta (como "la tela pica" o "talla pequeña") en segundos.

4. Resultados obtenidos

Los resultados obtenidos a partir de los datos reflejan la efectividad del paquete en la extracción de patrones e insights relevantes, y ayudan a demostrar que el enfoque adoptado es adecuado para trabajar con grandes volúmenes de datos.

4.1. Perfil Demográfico: Edad Continua vs. Agrupada

Para comprender en profundidad el perfil del cliente, se contrastó la distribución de edad en su formato original continuo frente a la variable categórica generada durante el preprocesamiento:



Histogram de Edad (Izquierda)

- **Distribución:** El histograma de edades muestra una distribución casi normal con un ligero sesgo hacia la derecha.
- **Pico Principal:** La mayor concentración de clientes se sitúa entre los 30 y 50 años, con un pico notable alrededor de los 40 años.

Grupos de Edad (Derecha) Este gráfico de barras valida dos decisiones técnicas clave del proyecto:

- **Distribución:** Confirma que el algoritmo de *binning* (implementado en *DataLoader*) funcionó correctamente, agrupando a los usuarios en décadas (30-39, 40-49, etc.).
- **Visualización:** Demuestra la utilidad del parámetro `sort_by_index=True` en `plot_bar_chart`. Si se hubiera ordenado por frecuencia, los grupos habrían aparecido

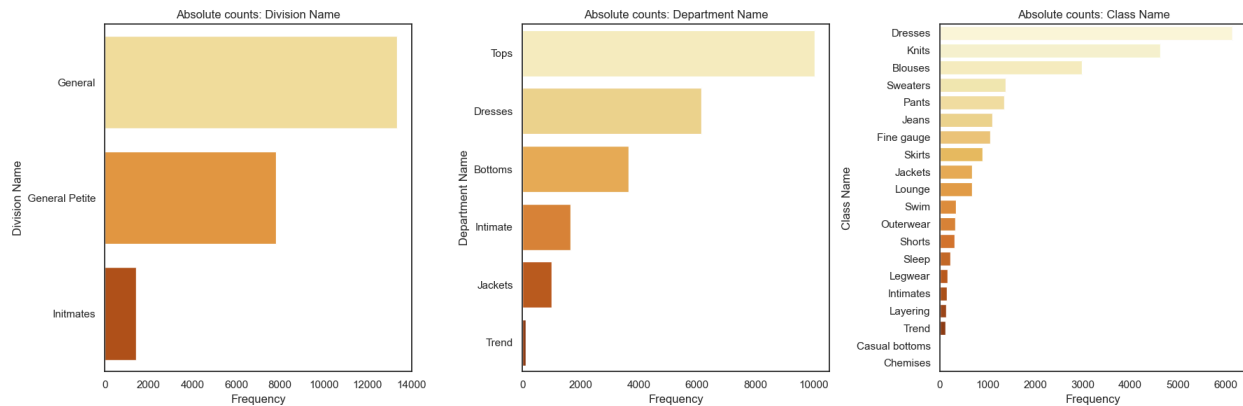
desordenados (ej. 30-39 primero, luego 40-49, luego 20-29). Al forzar el orden lógico, la tendencia generacional se lee de izquierda a derecha.

Insight de Negocio

Ambos gráficos confirman que el *Core Target* de la marca es la mujer profesional de **30 a 50 años** (estos dos grupos suman la gran mayoría de las ventas). La marca tiene muy poca penetración en el mercado de adolescentes, lo cual sugiere que el diseño y el tallaje están orientados a un cuerpo y estilo más maduro.

4.2 Distribución del Inventario por Categoría (División, Departamento y Clase)

Utilizando la función `plot_bar_chart` con orientación horizontal (`orientation='h'`), se desglosó el inventario en sus tres niveles jerárquicos para identificar los motores de venta:

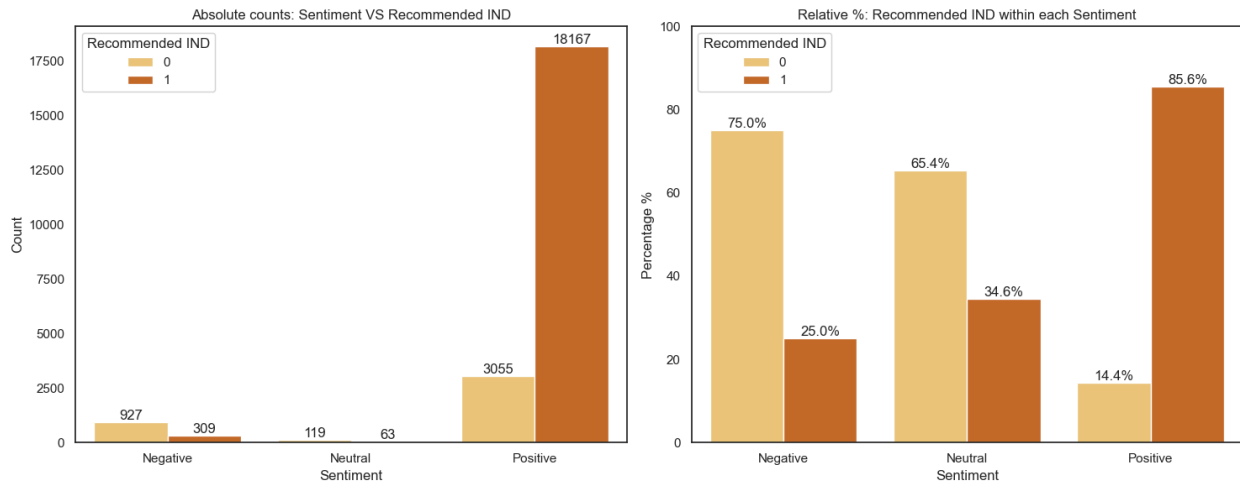


- **Dominio de la División "General":** El gráfico de la izquierda muestra que la división estándar ("General") representa la gran mayoría de las reseñas, duplicando casi el volumen de la división "General Petite". La división de ropa interior ("Intimates") ocupa un nicho menor.
- **Liderazgo de "Tops" y "Dresses":** A nivel de departamento (gráfico central), se observa que la tienda es fundamentalmente un destino para comprar partes de arriba (*Tops*) y vestidos (*Dresses*). Estos dos departamentos suman más del 60% del volumen total de interacciones.
- **El Nicho "Trend":** Es notable el volumen extremadamente bajo del departamento "Trend" (Tendencia). Esto sugiere que la marca funciona mejor con ropa clásica o básica que con artículos de moda experimental.
- **Granularidad Específica (Class Name)** El gráfico de la derecha ofrece el nivel más profundo de detalle.

El "**Best-Seller**": Aunque "*Tops*" es el departamento líder, la clase individual más vendida son los Vestidos (*Dresses*), seguidos de cerca por tejidos de punto (*Knits*) y blusas (*Blouses*).

4.3 Relación entre Sentimiento (VADER) y Recomendación del Cliente

Utilizando la función `plot_sentiment_vs_recommendations`, cruzamos la clasificación automática de sentimientos (*Negativo*, *Neutral*, *Positivo*) con la decisión final del cliente de recomendar o no el producto (*Recommended IND*).



Volumen de Reseñas (Izquierda)

Este gráfico de barras muestra la cantidad total de reseñas.

- **Predominio Positivo:** La gran mayoría de los comentarios son *Positivos* (más de 20,000). Esto es coherente con el alto *Rating* promedio de la tienda.
- **Escasez de Neutrales:** El algoritmo VADER clasifica muy pocas reseñas como "Neutrales", lo que indica que los clientes suelen usar un lenguaje con carga emocional clara (sea buena o mala).

Porcentaje de Recomendación (Derecha)

Aquí vemos qué porcentaje de clientes recomienda el producto dentro de cada categoría de sentimiento.

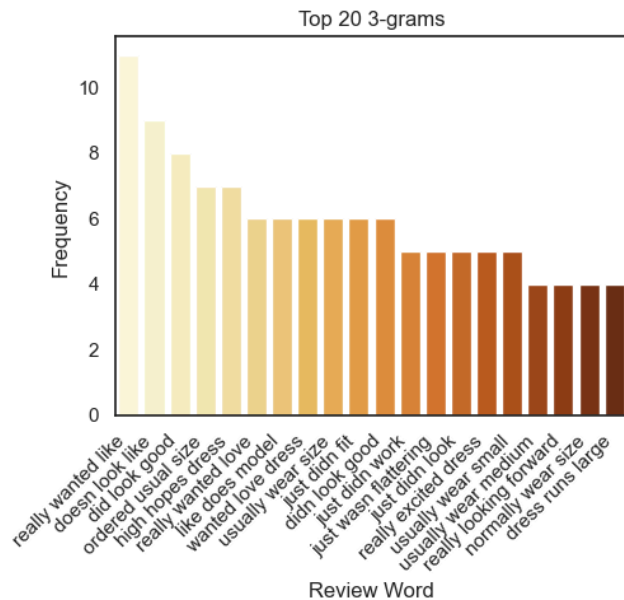
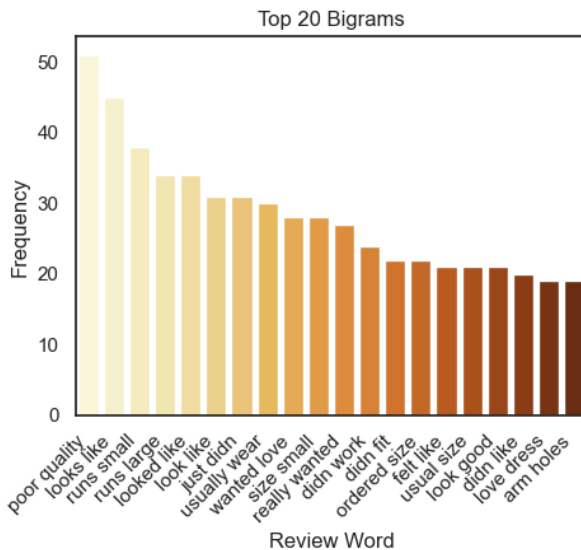
- **Validación del Modelo:** Existe una correlación muy fuerte.
 - El 86% de las reseñas clasificadas como Positivas recomiendan el producto.
 - El 75% de las reseñas Negativas NO lo recomiendan.

- Esto confirma que el algoritmo VADER está "entendiendo" bien el texto.
- **El Hallazgo Interesante (La "Disonancia"):**
 - Hay un 14% de reseñas que el algoritmo ve como "Positivas" pero donde el cliente NO recomienda el producto.
 - Interpretación: Esto suele ocurrir cuando el cliente es educado ("El vestido es precioso...") pero tiene un problema fatal ("...pero la cremallera se rompió al segundo día"). El algoritmo detecta palabras bonitas, pero la experiencia final fue mala.

El análisis de sentimiento coincide en gran medida con la recomendación del usuario, lo que valida la utilidad de la herramienta *TextAnalyzer*. Sin embargo, las discrepancias (ese 14%) nos recuerdan que el lenguaje humano es complejo y a veces contradictorio.

4.4 Radiografía de la Insatisfacción: WordCloud y Bigramas

Al cruzar la Nube de Palabras con el análisis de Bigramas (pares de palabras) y Trigrams, obtenemos una visión precisa de por qué los clientes dejan reseñas negativas:



[illegible]

El Caos de las Tallas (Sizing)

- **WordCloud:** La palabra "SIZE" llama la atención visualmente en la imagen.
- **Bigramas:** Las barras nos explican el problema real. Vemos tanto "*runs small*" (queda pequeño) como "*runs large*" (queda grande).
- **Insight:** El problema no es que la ropa sea pequeña por defecto, sino que el tallaje es inconsistente. Las clientas piden su talla habitual ("*usually wear*") y se encuentran con que la prenda no se ajusta a lo esperado, generando frustración.

La Brecha de Expectativas (Look & Feel)

- **WordCloud:** Palabras como "LOOK", "COLOR" son muy frecuentes.
- **Bigramas:** Aparecen frases reveladoras como "*looks like*" (se parece a...) y "*really wanted*" (realmente quería...).
- **Insight:** Estas frases denotan decepción emocional. La clienta ve la foto, se enamora ("*really wanted to love it*"), pero al recibir el producto, la realidad no coincide con la imagen de la web ("*looks like nothing like the picture*").

Calidad Percibida

-
- Mientras que en la nube destaca "FABRIC" (Tela), los bigramas son directos y duros: "poor quality" (mala calidad) es una de las quejas principales. No es solo un problema de estética, es un problema de materiales.

Conclusión

Los clientes no devuelven la ropa porque "no les guste el estilo", sino porque sienten que la marca rompió su promesa: la talla no es la estándar y mala calidad de la tela.

5. Problemas y Soluciones

El desarrollo de un paquete de software modular presentó varios desafíos técnicos significativos. A continuación, se detallan los más relevantes y cómo se resolvieron.

5.1. Visualización Dual (Jupyter vs. Streamlit)

El Problema: Las funciones de visualización originalmente utilizaban *plt.show()*, lo cual funciona en Jupyter Notebooks pero no renderiza nada en la interfaz web de Streamlit. Además, Streamlit requiere recibir el objeto *Figure* explícitamente.

La Solución: Se refactorizaron todos los métodos de las clases de visualización (*DataVisualizer* y *TextAnalyzer*) para que retornaran el objeto *fig* (figura de Matplotlib) en lugar de mostrarlo directamente.

- *Código implementado:* return *fig* al final de cada función.
- *Resultado:* Esto permitió usar *display(fig)* en Jupyter y *st.pyplot(fig)* en Streamlit con el mismo código base, aumentando la versatilidad del paquete.

5.2. Lógica de Agrupación de Datos (Binning)

El Problema: La creación de grupos de edad (*Age Group*) inicialmente requería múltiples sentencias *if-else*, lo que hacía el código extenso, lento y difícil de mantener si los rangos cambiaban.

La Solución: Se implementó una solución matemática basada en la "división de piso" (*floor division*).

- *Fórmula:* $(\text{Edad} // 10) * 10$.

-
- Esto automatizó la creación de etiquetas (ej. "30-39", "40-49") en una sola línea de código vectorizada con Pandas, mejorando la eficiencia y la limpieza del código.

Uso de Herencia (Inheritance): Se decidió que la clase *TextAnalyzer* herede de *DataVisualizer*.

- El análisis de texto a menudo culmina en visualizaciones (nubes de palabras, gráficos de barras de sentimientos). Al heredar, *TextAnalyzer* puede reutilizar métodos de graficado de su clase padre sin duplicar código, cumpliendo con el principio DRY (*Don't Repeat Yourself*).

Encadenamiento de Métodos (Method Chaining): En la clase *Dataloader*, los métodos de limpieza retornan *self* en vez de *self.df*.

- Esto permite una sintaxis fluida y legible en el script principal (ej. *loader.load().clean().process()*), similar a la filosofía de librerías modernas como Pandas o jQuery.

6. Conclusiones y Trabajo Futuro

6.1. Conclusiones

El proyecto ha cumplido exitosamente con su objetivo principal: desarrollar una herramienta analítica robusta y modular para el análisis de reseñas de *e-commerce*.

1. **Valor de la Arquitectura Modular:** La separación en clases (*Loader*, *DataVisualizer*, *TextAnalyzer*) ha demostrado ser superior a los scripts tradicionales. Permite aislar errores rápidamente y facilita la lectura del código.
2. **Poder del NLP:** Se ha confirmado que el análisis de texto aporta una dimensión crítica que las simples calificaciones de estrellas no pueden capturar. Identificar *por qué* un producto falla (ej. "tela barata") es más valioso para el negocio que saber simplemente que falló.
3. **Experiencia de Usuario:** Streamlit se ha validado como una herramienta excelente para prototipado rápido de aplicaciones de datos, permitiendo transformar análisis técnicos en interfaces accesibles para usuarios de negocio.

6.2. Trabajo Futuro

A pesar del éxito del proyecto, existen áreas claras para futuras iteraciones y mejoras:

-
1. **Modelos de NLP más avanzados:** Actualmente se utiliza VADER (basado en reglas). Una mejora significativa sería implementar modelos basados en Transformers (como BERT o RoBERTa) para capturar mejor el sarcasmo y el contexto complejo en las reseñas.
 2. **API Rest:** Desacoplar el frontend (Streamlit) del backend. Exponer las clases *DataLoader* y *TextAnalyzer* a través de una API (usando FastAPI o Flask) permitiría que otros sistemas de la empresa consumieran los análisis en tiempo real.

7. Referencias

Agarap, A. F. (2018). *Women's E-Commerce Clothing Reviews*. Kaggle.

<https://www.kaggle.com/datasets/nicapotato/womens-ecommerce-clothing-reviews>

Atif Ali AK. (2025). *Twitter Sentiment Analysis | LSTM |98% Accuracy*. Kaggle.

<https://www.kaggle.com/code/atifaliak/twitter-sentiment-analysis-lstm-98-accuracy>

Austin Chia. (2024). *How to Make a Seaborn Histogram: A Detailed Guide*. datacamp.

<https://www.datacamp.com/tutorial/how-to-make-a-seaborn-histogram>

Bothma, J. (2024). *Seaborn Heatmaps: A Guide to Data Visualization*. DataCamp.

<https://www.datacamp.com/tutorial/seaborn-heatmaps>

García-Ferrerira. (2025). *Como crear un analizador de sentimientos de las noticias para la bolsa*.

García-Ferrerira.

<https://www.garcia-ferreira.es/como-crear-un-analizador-de-sentimientos-para-la-bolsa/>

GeeksforGeeks. (2025). *Seaborn.barplot() method in Python*. GeeksforGeeks.

<https://www.geeksforgeeks.org/data-visualization/seaborn-barplot-method-in-python/>

Graph Gallery. (2024). *Wordcloud*. Python Graph Gallery.

<https://python-graph-gallery.com/wordcloud/>

Lakshmipathi N. (2021). *Sentiment Analysis of IMDB Movie Reviews*. Kaggle.

<https://www.kaggle.com/code/lakshmi25npathi/sentiment-analysis-of-imdb-movie-reviews>

Mike Chirico. (2016). *Quick Look - Seaborn, WordCloud*. Kaggle.

<https://www.kaggle.com/code/mchirico/quick-look-seaborn-wordcloud>

NICAPOTATO. (2019). *Guided Numeric and Text Exploration E-Commerce*. Kaggle.

<https://www.kaggle.com/code/nicapotato/guided-numeric-and-text-exploration-e-commerce#2.-Univariate-Distribution->

Pandas. (2026). *pandas.DataFrame — pandas 2.3.3 documentation*. Pandas.

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>

Ranjith kumar G. (2020). *NLP with Women Clothing reviews*. Kaggle.

<https://www.kaggle.com/code/granjithkumar/nlp-with-women-clothing-reviews/notebook>

Seaborn. (2024). *seaborn.heatmap — seaborn 0.13.2 documentation*. Seaborn.

<https://seaborn.pydata.org/generated/seaborn.heatmap.html>

Anexo: Registro de Prompts de IA

La herramienta de IA utilizada en este proyecto ha sido Gemini 3 (link: <https://gemini.google.com/app>).

Dado el elevado número de interacciones realizadas durante el desarrollo, diseñé una consulta para extraer el historial de interacciones con el siguiente prompt: *'Genera una lista de prompts escritos en este chat. Si alguno está en otro idioma, tradúcelo al español'*.

Una vez obtenido el listado, realicé una clasificación manual en siete bloques principales: Visualización, Pandas, NLP, Manejo de Excepciones, Refactorización, Documentación y Estructura.

1. Visualización de Datos (Matplotlib, Seaborn y WordCloud)

Dudas específicas sobre cómo generar gráficos.

1.1. Histogram:

- "Para el dataset de Women's E-Commerce Clothing Reviews, quiero hacer un histograma de la columna de puntuación (Rating). ¿Cuántos bins debería configurar?"
- "Al usar `viz.plot_histogram(column_name='Class Name')`, los nombres se superponen. ¿Cómo puedo mejorarlo?"
- Las líneas de la cuadrícula blanca son un poco feas. ¿Cómo puedo cambiarlas?

1.2. Pie Chart:

- "El código del gráfico de pastel no me funciona, ¿me puedes mirar por qué?" (El error de la coma en la lista de colores).
- "en la función `plot_general_pie_chart`, cuando el `ax= None`, me sale una frase así en jupyter: `close() argument must be a Figure...`"
- La combinación de colores no me gusta, ¿puedes recomendar colores que vayan bien con el azul?"

1.3. Heatmap:

- "¿Puedes darme algunos otros ejemplos específicos sobre cómo dibujar mapas de calor (Heatmaps)?"
- "`plt.figure(figsize = (5,5))`... ¿Cuál es el uso de este código?"
- "No entendí qué hace la función `plt.tight_layout()`."

-
- Para el conjunto de datos "Reseñas de ropa de comercio electrónico para mujeres", ¿Qué datos me recomiendas para crear un mapa de calor?
 - "¿Me puedes dar algún consejo para mejorar mi código de `plot_heatmap_crosstab`?"

1.4. Wordcloud

- "Por favor, combínalo con este código y explícame cómo usar la nube de palabras (Wordcloud)..."
- "¿Es cierto que solo hizo una estadística de nubes de palabras de todos los comentarios? No distinguió si eran positivos o negativos..."
- "¿Puedes darme otro ejemplo sencillo para explicar cómo usar la nube de palabras?"
- "mpl.rcParams... ¿Qué significa esta parte?"
- "Si voy a escribir una clase principalmente para dibujar, ¿debo configurar estos parámetros dentro de la clase?"
- "No entendí `sum_words[0, idx]`. ¿Por qué hay un 0 ahí?"
- "me puedes mirar el error de esta función? `def plot_wordcloud...`"

1.5. N-grams

- "¿Puedes presentarme qué son los n-grams? Si quiero analizar esta base de datos específica, ¿cómo debería usarlos?"
- "¿Las librerías `nlk.util.ngrams` y `collections.Counter` también sirven para hacer análisis de n-grams?"
- "¿Cómo instalar la librería `vaderSentiment`?"

1.6. Streamlit

- Elección de Framework: "He desarrollado por mi cuenta un paquete principalmente para el análisis de datos de Women's E-Commerce Clothing Reviews. Este paquete sirve para limpiar datos y generar gráficos. Quiero hacer un dashboard para mostrar estos gráficos, ¿puedo usar Django para desarrollarlo? Además de Django, ¿qué otros frameworks me recomiendas?"
- Prueba de Streamlit: "Quiero probar Streamlit, ¿puedes darme un ejemplo?"
- Compatibilidad con Jupyter: "Pero mi paquete no solo necesita mostrarse en Streamlit, también necesito generar gráficos en Jupyter. Si lo cambio a `return fig`, ¿seguirá funcionando en Jupyter?"
- Gráficos duplicados: "¿Por qué cuando ejecuto esto en Jupyter aparecen dos gráficos exactamente iguales?"

-
- Filtros dinámicos: "Para la página web que muestra Streamlit, ¿cómo puedo añadir filtros para que los gráficos sean dinámicos?"
 - Cuando ejecuto Streamlit, me sale el warning: “`use\_container\_width` will be removed after 2025-12-31. For `use\_container\_width=True`, use `width='stretch'`. For `use\_container\_width=False`, use `width='content'`. 2026-01-01 18:34:18.395 Please replace `use\_container\_width` with `width`”. ¿Cómo debería cambiar el código en mi Streamlit?
 - `close()` argument must be a Figure, an int, a string, or None, not <class 'matplotlib.axes._axes.Axes'> me puedes mirar por qué me sale este error?

2. Manipulación de Datos y Pandas

Preguntas sobre la sintaxis específica de Python y Pandas para limpiar y manejar datos.

- "Para esta base de datos... ¿Es necesario analizar el título y el cuerpo (Review Text) por separado? ¿O es mejor que los unamos a una columna?"
- "Si lo que quiero es el dataframe antes de que esta función lo limpie, ¿es necesario que use `df.copy()`?"
- "En la línea `count_rec = self.df[...].shape[0]`, ¿qué significa `shape[0]`? Explícamelo."
- `counts = self.df[column_name].value_counts().sort_index()` esta función saca un pandas series?
- `data = counts.values.tolist()` hace falta poner `.tolist()`, ¿qué tipo de datos es `counts.values`?
- La línea `display(self.df.nunique())` no se muestra. ¿Hay alguna otra forma de mostrar el valor único de una columna?
- `data.values` Por favor, dame algunos ejemplos más para explicar este uso."

'Unnamed: 0'

- en el dataset hay una columna que se llama 'Unnamed: 0', son 0, 1, 2, 3, estos números que no sirve para nada, lo puedo directamente drop esta columna, no?
- `self.df.drop(columns=['Unnamed: 0'], inplace=True)` para qué sirve el `inplace=True`
- `self.df = self.df.drop(columns=['Unnamed: 0'], inplace=True)` is ok?
- `self.df.drop(columns=['Unnamed: 0'], inplace=True)` por que va a return None, no he entendido esta parte

display VS print

- "Si necesito usar `display`, ¿tengo que importar alguna librería?"

-
- "¿Cuál es la diferencia entre display y print?"

3. Teoría de Machine Learning y NLP (Conceptos)

- "Todavía no entiendo la parte del entrenamiento del modelo, ¿necesitamos añadir manualmente las etiquetas de entrenamiento?"
- "Si ya tengo la calificación por estrellas, ¿qué sentido tiene juzgar si los comentarios individuales son positivos o negativos?"
- "Usando las 'calificaciones por estrellas' existentes vs. generando etiquetas con VADER... ¿cuál recomiendas?"
- "¿Cuál es el sentido de entrenar el modelo (si ya usas TextBlob)?"
- "¿Puedes explicarme cómo se entrena Naive Bayes? ¿Cuál es el principio?"
- "¿Cuánto tiempo necesito para aprender LSTM? ¿Puedes enseñarlo de forma sencilla?"
- "¿Qué método utiliza este documento de código para entrenar el modelo? (Naive Bayes vs LSTM)".

4. Manejo de Excepciones

- "me puedes mirar si esta forma de escribir personalizados errores es correcta? (DataFrameNoneError...)"
- "cómo puedo poner el column_name como parámetro para el error ColumnNameNotExistError"
- "cómo puedo mejorar esta función para que sea más robusta? por ejemplo, poner .strip()?"
- "...así funcionará?" (Revisión de lógica .lower())
- "ok, decido no poner lo de lower, me puedes mirar ahora?"
- "necesito poner if column_name is None? aparte de if not column_name?"
- "ModuleNotFoundError: No module named 'data_visualizer'" y solicitud de explicación.

5. Refactorización y Patrones de Diseño (Clean Code)

Dado que he puesto demasiado largo y complicado los métodos y he repetido varias veces trozos de código, pedí a la IA que me recomendara formas para separar las responsabilidades de cada parte.

-
- "estoy pensando si existe la posibilidad de combinar la funcion plot_bar_chart con la funcion plot_sentiment_vs_recommendations..."
 - "estoy pensando que quizá esta función es demasiado largo, hace falta cortar a funciones más cortas? def plot_top_ngrams..."
 - "hace falta separar esta función? def plot_wordcloud..."
 - "Esta parte será mejor modificarla así? if sentiment is not None:..."
 - "necesito tu ayuda para separar esta función en 3 funciones... (plot_sentiment_vs_recommendations)"
 - "ayúdame a separar esta función: def process_ecommerce_data(self):"

Gestión del Canvas (DRY): "esta función se puede separar a varias? def plot_histogram..."

- "en esta función puedo aplicar la misma regla? def plot_bar_chart..."
- "y esta función también, no? def plot_one_heatmap..."
- "y también esta función, no? def plot_recommendation_rate_heatmap..."

Separación de Responsabilidades (Cálculo y Visualización):

"plot_recommendation_rate_heatmap() esta función se puede separar aún más o es mejor mantener la estructura de ahora?"

- "se puede mejorar más esta función? plot_one_heatmap, hay que separar alguna responsabilidad?"
- "¿me puedes dar consejos para separar las responsabilidades de esta función? def plot_general_pie_chart..."
- "también necesito ayuda para separar las responsabilidades de esta función: def plot_bar_chart..."

6. Documentación

Prompts para generar Docstrings

- "can you help me to put the doc string of this function in English? def _validate_data"
- "me puedes generar el docstring para las funciones ['plot_bar_chart', 'plot_correlation_heatmap', ...]"
- "me puedes generar el docstring para las funciones ['clean_data', 'load_data', ...]"
- "me puedes ayudar a escribir docstrings para las funciones de este documento en inglés? (TextAnalyzer)"

Prompts para escribir la memoria

- "me puedes ayudar a escribir un breve análisis sobre este chart?" (Histograma de Edad)
- "me puedes ayudar a escribir un breve análisis sobre este chart? la función que he usado es esta `text_viz.plot_sentiment_vs_recommendations...`"
- "me puedes ayudar a escribir un breve análisis sobre este chart? la función que he usado es esta: `text_viz.plot_wordcloud...`"
- "si relaciono el anterior wordcloud junto con este bi-grams de reseñas negativas, qué insight puedo sacar de estos dos charts?"

7. Estructura del Proyecto y Despliegue

- Integración Web (Django): "Tengo una idea: después de terminar este package, quiero usar el framework Django para hacer un dashboard de visualización. ¿Es posible?"
- Importación básica: "He creado la clase en `data_visualizer.py` e intento usarla en otro archivo, ¿cómo puedo importar esta clase?"
- Depuración de rutas: "Originalmente puse ambos archivos en una subcarpeta llamada 'module name', ahora los saqué todos y los puse en el proyecto principal y funciona. ¿Por qué sucede esto?"
- Importación avanzada: "Vale, si quiero importar archivos que no están en la misma carpeta, ¿cómo debo hacerlo?"
- Generar fotos: "estoy escribiendo un informe para desarrollar un paquete de python para analizar el dataset Women's E-Commerce Clothing Reviews, necesito tu ayuda para generar una foto para poner en la portada"

Mejora para el doc `main.py`

- "en este paquete, cómo debería poner el archivo de `main.py`?"
- "no debería poner algo simple como 'Main module' ...?"

UML diagrama

- "cómo genero el UML diagrama de todo este paquete?"
- "en este UML diagrama que me generaste antes, puedes añadir también la parte de Streamlit? (UML diagrama)"

READ.ME

-
- "sobre el readme.md, no tengo ni idea sobre cómo debería escribirlo, ¿me puedes dar un ejemplo?"
 - "Necesito tu ayuda para generar el proyecto estructural que aparece en readme.md..."
 - "Tengo una duda, el dataset que utilizo tiene que estar en una carpeta que se llama resources?..."

requirements_dev.txt

- "En este documento de requirements_dev.txt pone pip... necesito modificarlo?"
- "no hace falta poner pip==23.3.1...?"